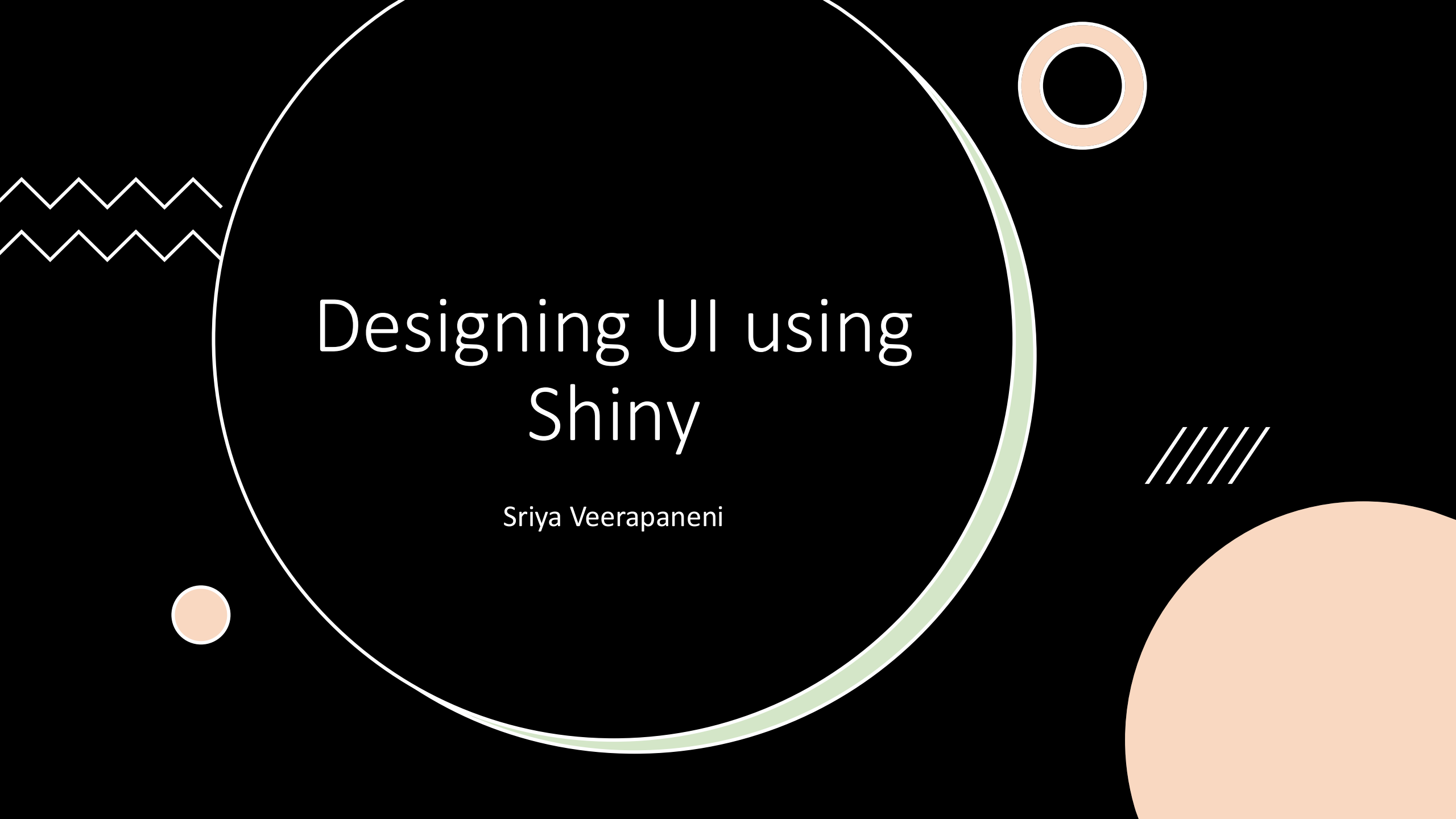




Designing UI using Shiny

Sriya Veerapaneni

What is UI



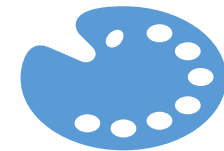
Visual designing of the
app



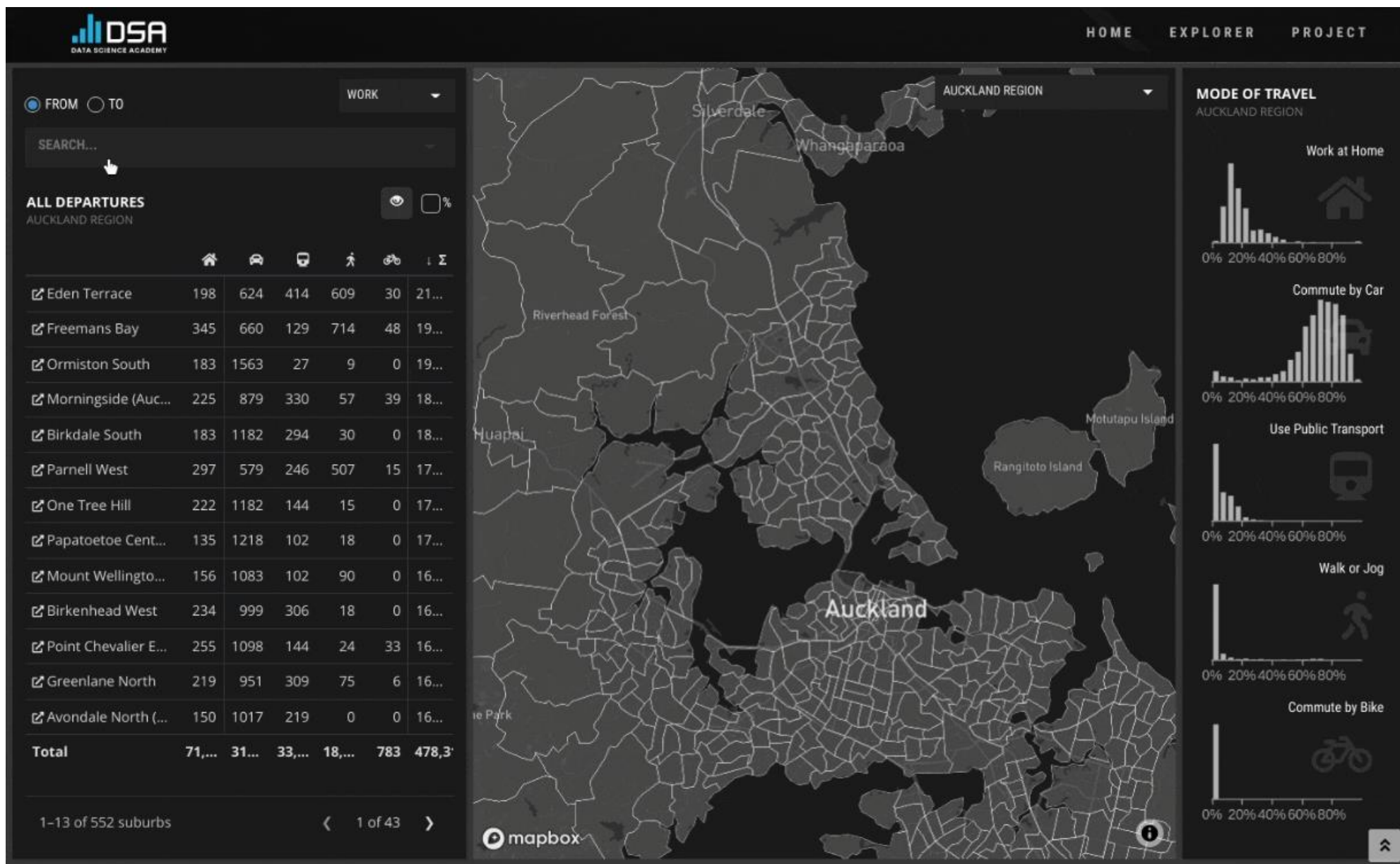
Defining all the elements
that go into the app



Altering layout and
graphs



Setting overall themes
and colors

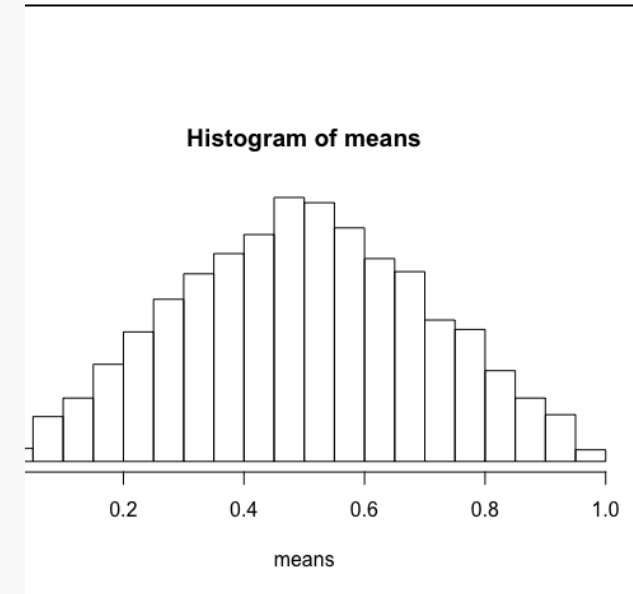


<https://nz-stefan.shinyapps.io/commute-explorer-2/>

Designing L

```
ui <- fluidPage(  
  titlePanel("Central limit theorem"),  
  sidebarLayout(  
    sidebarPanel(  
      numericInput("m", "Number of sample  
    ),  
    mainPanel(  
      plotOutput("hist")  
    )  
  )  
)  
server <- function(input, output, session)  
  output$hist <- renderPlot({  
    means <- replicate(1e4, mean(runif(in  
    hist(means, breaks = 20)  
  }, res = 96)  
}
```

```
html  
  
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>Central Limit Theorem</title>  
  <script src="https://cdn.plot.ly/plotly-latest.min.js"></script>  
  <style>  
    body {  
      font-family: sans-serif;  
      margin: 40px;  
    }  
    #histogram {  
      width: 800px;  
      height: 500px;  
    }  
  </style>  
</head>  
<body>  
  
  <h2>Central Limit Theorem</h2>  
  
  <label for="samples">Number of samples (m):</label>  
  <input type="number" id="samples" value="2" min="1" max="100">  
  <button onclick="simulate()">Simulate</button>  
  
  <div id="histogram"></div>  
  
  <script>  
    function simulate() {  
      const m = parseInt(document.getElementById("samples").value);  
      const nTrials = 10000;  
      let means = [];
```



Input functions in Shiny

Common structure:

`inputId` : Identifier

`label` : Human readable label

`value` : default value set to input

```
sliderInput("min", "Limit (minimum)", value = 50, min = 0, max = 100)
```

Input functions in Shiny

- `textInput()`
 - `numericInput()`
 - `selectInput()`
 - `sliderInput()`
 - `dateInput()`
- etc....

Action button

Action

Current Value:

[1] 0
attr("class")
[1] "shinyActionButtonValue" "integer"

See Code

Single checkbox

☒ Choice A

Current Value:

[1] TRUE

See Code

Checkbox group

☒ Choice 1
☐ Choice 2
☐ Choice 3

Current Values:

[1] "1"

See Code

Date input

2014-01-01

Current Value:

[1] "2014-01-01"

See Code

Date range

2025-07-17 to 2025-07-17

Current Values:

[1] "2025-07-17" "2025-07-17"

See Code

File input

Browse... No file selected

Current Value:

NULL

See Code

Numeric input

1

Current Value:

[1] 1

See Code

Radio buttons

☒ Choice 1
☐ Choice 2
☐ Choice 3

Current Values:

[1] "1"

See Code

Select box

Choice 1

Current Value:

[1] "1"

See Code

Slider

0 50 100

Current Value:

[1] 50

See Code

Slider range

0 25 75 100

Current Values:

[1] 25 75

See Code

Text input

Enter text...

Current Value:

[1] "Enter text..."

See Code

<https://shiny.posit.co/r/gallery/widgets/widget-gallery/>

Outputs in Shiny

- Placeholders defined inside the **UI** that can later be called in **Server**

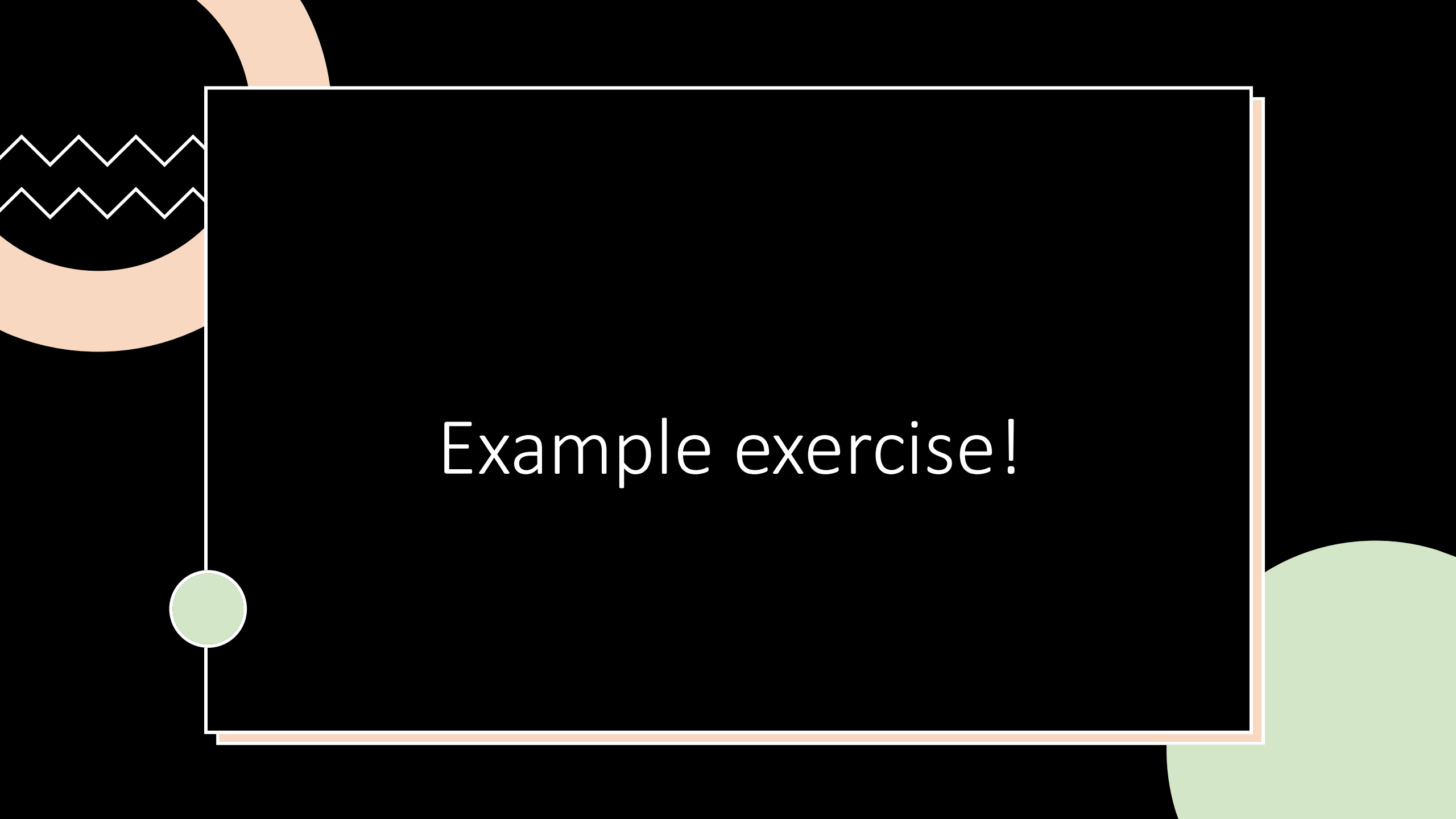
```
ui <- fluidPage(  
  textOutput("text"),  
  verbatimTextOutput("print")  
)  
server <- function(input, output, session) {  
  output$text <- renderText("hello!")  
  output$print <- renderPrint("hello!")  
}
```

hello!

```
[1] "hello!"
```


Outputs in Shiny

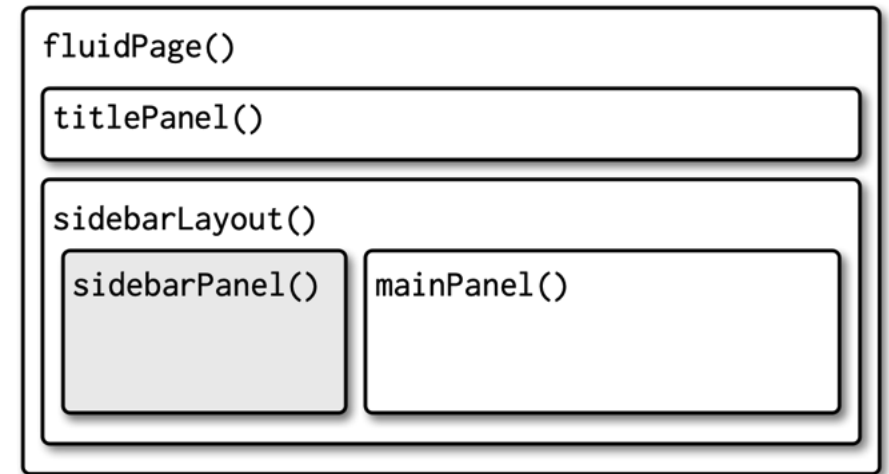
- `textOutput()`: `renderText()`
 - `verbatimTextOutput()`: `renderPrint()`
 - `tableOutput()`: `renderTable()`
 - `dataTableOutput()`: `renderDataTable()`
 - `plotOutput()`: `renderPlot()`
- etc...



Example exercise!

Layout: Sidebar Layout

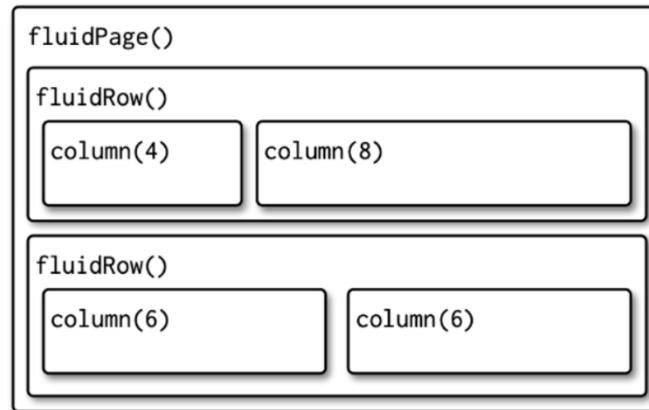
```
fluidPage(  
  titlePanel(  
    # app title/description  
  ),  
  sidebarLayout(  
    sidebarPanel(  
      # inputs  
    ),  
    mainPanel(  
      # outputs  
    )  
  )  
)
```



Custom Layouts in Shiny

- Under the hood `SidebarLayout()` is made of a flexible Multirow layout system. This can be used to make custom layouts.
- Within the `fluidPage()`, we can create rows with `fluidRow()`, and columns with `column()`

```
fluidPage(  
  fluidRow(  
    column(4,  
      ...  
    ),  
    column(8,  
      ...  
    )  
  ),  
  fluidRow(  
    column(6,  
      ...  
    ),  
    column(6,  
      ...  
    )  
  )  
)
```



- Each row is made of 12 columns.
- We can adjust column size inside the column function.

Layout: Sidebar Layout

```
library(shiny)
library(bslib)

# Define UI for application that draws a histogram
ui <- page_sidebar(

  title = "Hello Shiny!",

  sidebar = sidebar(
    sliderInput("bins", "Number of bins:", min = 1, value = 30, max = 50)
  ),

  plotOutput("distPlot")
)
```

```
# Define server logic required to draw a histogram
server <- function(input, output) {

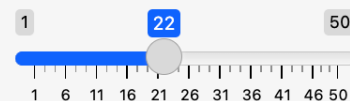
  output$distPlot <- renderPlot({
    # generate bins based on input$bins from ui.R
    x <- faithful[, 2]
    bins <- seq(min(x), max(x), length.out = input$bins + 1)

    # draw the histogram with the specified number of bins
    hist(x, breaks = bins, col = 'darkgray', border = 'white',
        xlab = 'Waiting time to next eruption (in mins)',
        main = 'Histogram of waiting times')
  })
}

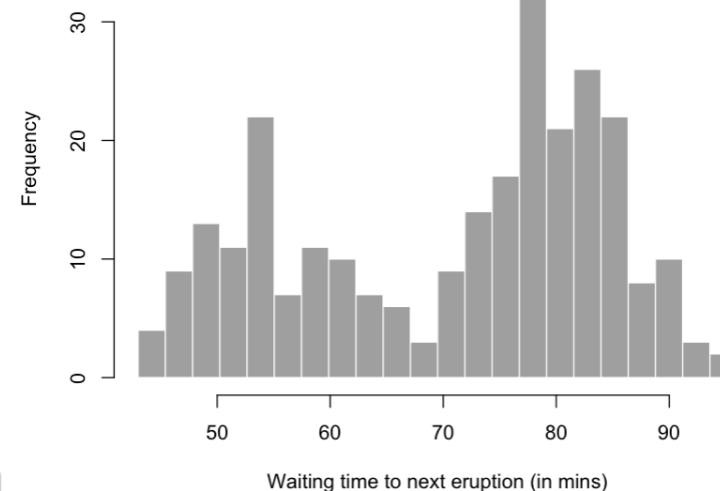
# Run the application
shinyApp(ui = ui, server = server)
```

Hello Shiny!

Number of bins:

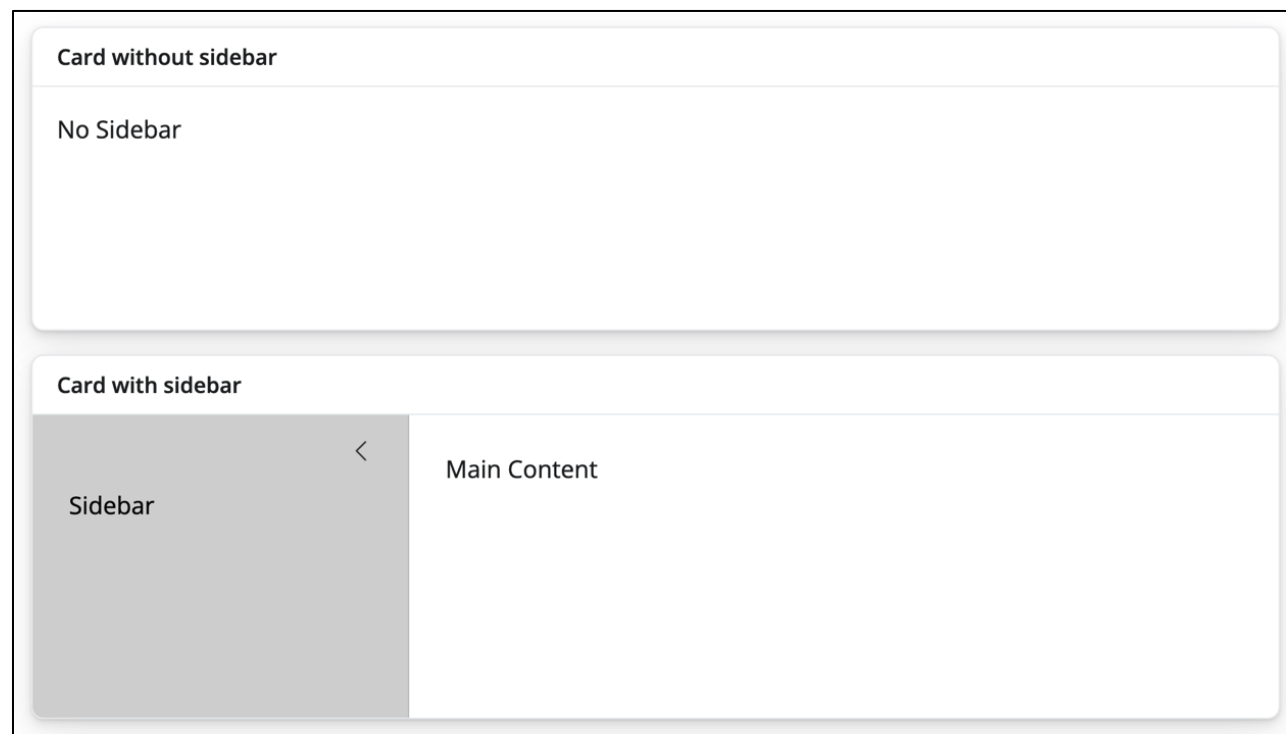


Histogram of waiting times



Layout: Cards

```
ui <- page_fillable(  
  
  card(  
    card_header("Card without sidebar"),  
    "No Sidebar"  
  ),  
  
  card(  
    card_header("Card with sidebar"),  
    layout_sidebar(  
      sidebar = sidebar(  
        bg = "lightgrey",  
        "Sidebar"  
      ),  
      "Main Content"  
    )  
  )  
)
```



Layout: Tabs

`tabsetPanel( id = "", tabPanel("panel a", "A"), ...)`

```
ui <- fluidPage(  
  sidebarLayout(  
    sidebarPanel(  
      textOutput("panel")  
    ),  
    mainPanel(  
      tabsetPanel(  
        id = "tabset",  
        tabPanel("panel 1", "one"),  
        tabPanel("panel 2", "two"),  
        tabPanel("panel 3", "three")  
      )  
    )  
  )  
)  
server <- function(input, output, session) {  
  output$panel <- renderText({  
    paste("Current panel: ", input$tabset)  
  })  
}
```



Layout: Tabs

NavbarPage(): Horizontal tabs +
navbarMenu() to set dropdowns

```
ui <- navbarPage(  
  "Page title",  
  tabPanel("panel 1", "one"),  
  tabPanel("panel 2", "two"),  
  tabPanel("panel 3", "three"),  
  navbarMenu("subpanels",  
    tabPanel("panel 4a", "four-a"),  
    tabPanel("panel 4b", "four-b"),  
    tabPanel("panel 4c", "four-c")  
  )  
)
```



navlistPanel() makes vertical tabs

```
ui <- fluidPage(  
  navlistPanel(  
    id = "tabset",  
    "Heading 1",  
    tabPanel("panel 1", "Panel one contents"),  
    "Heading 2",  
    tabPanel("panel 2", "Panel two contents"),  
    tabPanel("panel 3", "Panel three contents")  
  )  
)
```



Layout: Tabs

To add multiple pages to an app, change `page_sidebar()` to `page_navbar()`

```
ui <- page_navbar(  
  title = "My App",  
  bg = "#2D89C8",  
  inverse = TRUE,  
  nav_panel(title = "One", p("First page content.")),  
  nav_panel(title = "Two", p("Second page content.")),  
  nav_panel(title = "Three", p("Third page content.")),  
  nav_spacer(),  
  nav_menu(  
    title = "Links",  
    align = "right",  
    nav_item(tags$a("Posit", href = "https://posit.co")),  
    nav_item(tags$a("Shiny", href = "https://shiny.posit.co"))  
  )  
)
```



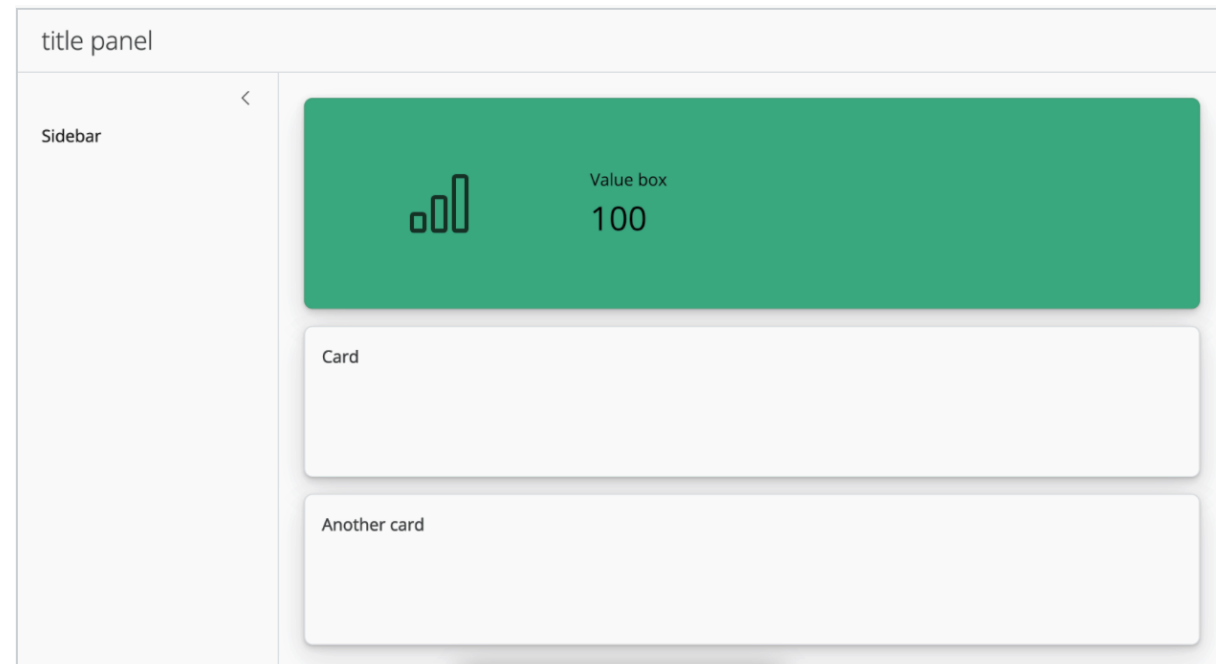
Layout: Tabs

- Use **bslib** to customize the layout:
- **nav_panel()** - Use to supply the title and content for each page.
- **nav_spacer()** - Acts as a flexible divider that pushes all subsequent items (like the "Links" menu) to the far right of the navigation bar.
- **nav_menu()** - Groups multiple items into a single dropdown menu titled "Links".
- **nav_item()** - Used here to wrap external hyperlinks so they appear as clickable elements within the dropdown menu.

```
ui <- page_navbar(  
  title = "My App",  
  bg = "#2D89C8",  
  inverse = TRUE,  
  nav_panel(title = "One", p("First page content.")),  
  nav_panel(title = "Two", p("Second page content.")),  
  nav_panel(title = "Three", p("Third page content.")),  
  nav_spacer(),  
  nav_menu(  
    title = "Links",  
    align = "right",  
    nav_item(tags$a("Posit", href = "https://posit.co")),  
    nav_item(tags$a("Shiny", href = "https://shiny.posit.co"))  
  )  
)
```

Layout: Value boxes

```
ui <- page_sidebar(  
  title = "title panel",  
  sidebar = sidebar("Sidebar"),  
  value_box(  
    title = "Value box",  
    value = 100,  
    showcase = bsicons::bs_icon("bar-chart"),  
    theme = "teal"  
  ),  
  card("Card"),  
  card("Another card")  
)
```

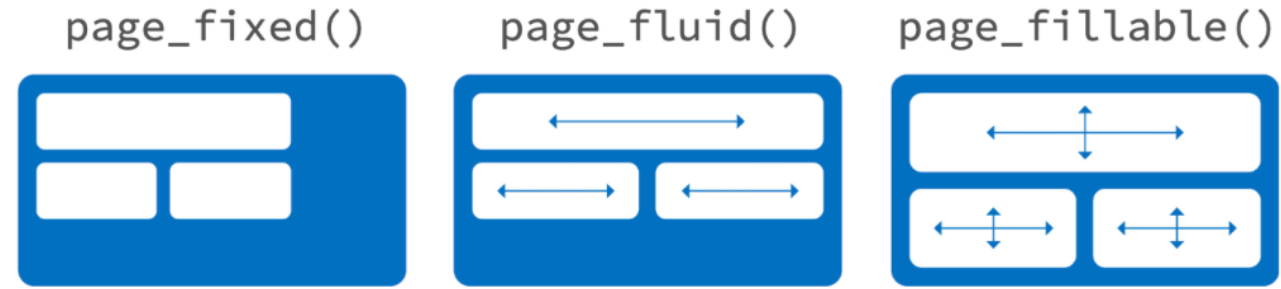


Layout: Filling Layout

page_fixed(): places components into a grid that occupies a fixed width of 940 pixels by default.

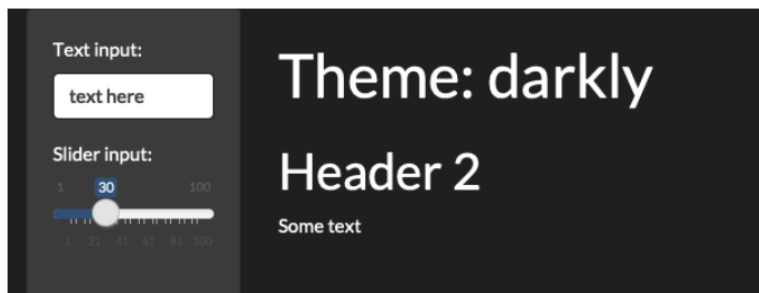
page_fluid(): places components into a grid that always occupies the full *width* of the web page, and it re-sizes its components dynamically as the *width* of the page changes.

page_fillable(): places components into a grid that always occupies the full *width and height* of the web page, and it re-sizes its components dynamically as the *width and height* of the page changes.



Themes

```
ui <- fluidPage(  
  theme = bslib::bs_theme(bootswatch = "darkly"),
```



Theme: flatly
Header 2

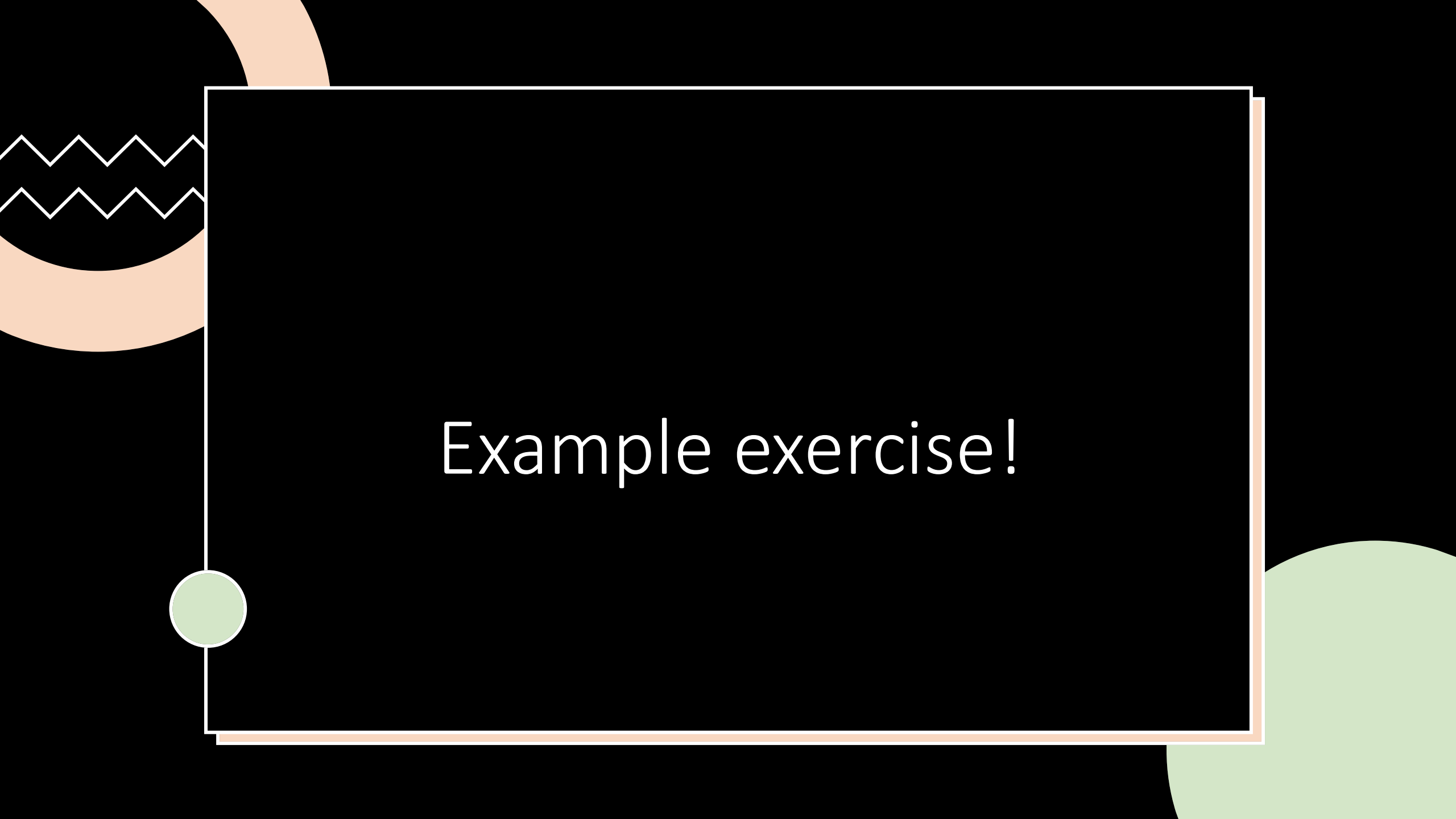
Theme: united
Header 2

Custom themes

```
theme <- bslib::bs_theme(  
  bg = "#0b3d91",  
  fg = "white",  
  base_font = "Source Sans Pro"  
)
```

Note: Use **thematic::thematic_shiny()** command inside the server function to match the plot themes to the app theme

<https://rstudio.github.io/shinythemes/>



Example exercise!